

Lexical Analysis: Problem Set

One exercise per method, one per lexing concept, and true/false questions

How to use this set

Part A has one exercise per method of Part II of *Lexical Analysis: The Complete Guide* (Methods 1 to 7). Part B has one exercise per lexing concept of Parts I and III: tokens and positions, hand-written lexers, regular expressions for tokens, conflict rules, scanning with pushback, lexer states, flex, and deriving a token pattern from its DFA. Part C is a set of true/false statements. A version with full solutions under every question is in *Lexical_Analysis_Problem_Set_Solutions.pdf*.

All grammars and expressions here are new: none is the running example of the guide or a problem from Problem Sets 2 to 4. Every answer was checked by program or by running `flex`. Difficulty: ★ direct, ★★ several steps, ★★★ proof or design.

Part A: The automata methods

A1. Thompson's construction (Method 1) (★★)

Let $r = (a \mid bc)^* b$.

(a) Parse r and count its leaves, unions, stars and concatenations.

(b) Build $N(r)$ with convention B (concatenation merges states). List the states and edges, and give the initial and final state.

(c) Check the counts against the formula, check property (P3), and give the counts under convention A.

A2. Simulating an NFA (Method 2) (★)

The NFA N over $\{a, b\}$ has start state 0, accepting state 5, and edges

$$0 \xrightarrow{\varepsilon} 1, \quad 0 \xrightarrow{\varepsilon} 4, \quad 1 \xrightarrow{a} 2, \quad 2 \xrightarrow{b} 3, \quad 3 \xrightarrow{\varepsilon} 0, \quad 4 \xrightarrow{a} 5, \quad 4 \xrightarrow{b} 6, \quad 6 \xrightarrow{b} 5.$$

(a) Simulate N on `abbb`, `aba` and `abab`. Give the state set after each character and the verdict.

(b) Describe $L(N)$ with a regular expression.

A3. Subset construction (Method 3) (★★)

Use the NFA N of A2.

(a) Run the subset construction. Name the DFA states A, B, ... in order of discovery and give the transition table and the accepting states.

(b) Two of these states are equivalent. Which, and why can you see it without the table-filling algorithm?

A4. Minimisation by table filling (Method 4) (★★)

A DFA over $\{a, b\}$ with start A and accepting state E:

	A	B	C	D	E	F
a	B	B	D	A	D	D
b	C	C	C	E	F	C

(a) Fill the distinguishability table: Rule 1, then sweeps of Rule 2. For each marked pair give the sweep and a witness string.

(b) Give the minimal DFA and describe its language.

A5. State elimination, DFA to regex (Method 5) (★★)

The minimal DFA from A3: states P (start), B , C , E ; accepting B and E ; edges $P \xrightarrow{a} B$, $P \xrightarrow{b} C$, $B \xrightarrow{b} P$, $C \xrightarrow{b} E$ (all other moves go to the dead state).

(a) Add a final state f and eliminate C , then E , then B . Give the resulting regular expression.

A6. Brzowski derivatives (Method 6) (★★)

Let $R = (ab \mid b)^*$.

(a) Compute $\partial_a R$ and $\partial_b R$, then the derivatives of every new expression, until no new expression appears. Mark which are nullable.

(b) Draw the resulting DFA, test **bab** and **aab**, and describe the language in words.

A7. Lower bounds and non-regularity (Method 7) (★★★)

(a) Let L_4 be the strings over $\{a, b\}$ whose number of a 's is divisible by 4. Prove that every DFA for L_4 has at least 4 states, and give a DFA with exactly 4.

(b) Prove that $L = \{a^n b^n : n \geq 0\}$ is not regular.

(c) Explain in two sentences what (b) says about comments that may be nested.

Part B: Lexing concepts

B1. Tokens, attributes and positions (★)

A lexer for a C-like language skips blanks and `//` comments. Integers are `[0-9]+`, floats follow decimal `("e"|"E") ("+"|"-"?) digit+ or decimal`, identifiers are `[a-z][a-z0-9]*`, and if is a keyword. The input is (columns start at 1):

```
1  if (x>=10) y = x * 2.5e-1;  
2  s = "hi\n"; // done
```

(a) List every token with its class, its attribute (if any), and its line:column.

(b) How many tokens are there? What is the difference between the string token's lexeme and its attribute? Why is `>=` one token?

B2. A hand-written lexer bug (★)

```
int n = 0, c;  
while ((c = input.read()) >= '0' && c <= '9')  
    n = n * 10 + (c - '0');  
return new Token(INT, n);
```

(a) Trace this code on `12+3`. Which tokens does the lexer produce? Why does `12 + 3` seem to work?

(b) Fix it.

B3. Writing token patterns (★★)

(a) Write a regular expression for binary literals: `0b` or `0B`, then binary digits, where single underscores may appear *between* digits (as in `0b1010_0101`). Give four strings it must reject.

(b) Write a regular expression for 24-hour times `HH:MM` (`00:00` to `23:59`). Test it on `19:07`, `24:00`, `7:30` and `12:60`.

B4. Longest match and rule order (★★)

A flex specification lists, in this order: `"=="` EQ, `"="` ASSIGN, `">="` ARROW, `[a-z]+` ID, `"else"` ELSE, `[0-9]+` INT, `[0-9]+ "." [0-9]+` FLOAT, `"."` DOT, and a rule skipping blanks.

(a) Give the token stream for `else elsewhere=>x==3.14.5`.

(b) What does flex report about this specification, and how do you fix it?

B5. Scanning with pushback (★★)

Rules, in order: T1 **ab**, T2 **abcd**, T3 **c**, T4 **b**. A read counts every character fetched, including the one on which the scanner gets stuck; reaching the end of the input is not a read.

(a) Draw the combined DFA with its token labels.

(b) Scan **abcabcd**. Give each token, the number of reads it costs, and its pushback. Give the total number of reads.

(c) What is the maximum pushback of this specification, over all inputs?

B6. Lexer states: indentation and nested comments (★★)

(a) Give the **INDENT** and **DEDENT** tokens, with the line that triggers each, for this Python file. Then say what happens if line 3 of a second file is indented by 2 spaces after a line indented by 4 (lines: 0, 4, 2 spaces).

```
1 if x:
2     y = 1
3     if y:
4         z = 2
5
6     w = 3
7 v = 4
```

(b) A lexer handles comments with a depth counter: `/*` increments it (entering **COMMENT** mode at depth 1), `*/` decrements it, and the lexer leaves **COMMENT** mode at depth 0. Trace the depth on `/* a /* b */ c */ d */`. Which tokens follow? What would a two-state **COMMENT** lexer without a counter produce?

B7. Predicting flex output (★★)

```
%%
x+y?    printf("<1:%s>", yytext);
xy      printf("<2>");
y+      printf("<3:%d>", (int) yyleng);
x*z     printf("<4:%s>", yytext);
.|\\n   printf("[%s]", yytext[0] == '\\n' ? "NL" : yytext);
```

(a) Predict the output on `xyyyzxx`, a newline, then `yxw`. Is rule 2 ever used?

B8. A token pattern from its DFA (★★★)

In SQL and Pascal, a string is written in single quotes, and a quote inside the string is written twice: `'it's'` is the string `it's`. Strings cannot contain a raw newline.

(a) Draw the DFA over the classes `'`, newline and *other*, then read off a regular expression.

(b) With a rule for blanks and a one-character error rule, what does flex produce on 'it's' 'x', on ' ' and on 'ab'? Explain the last one.

Part C: True or false

Decide, and justify in one line.

- C1. A DFA may have ε -transitions.
- C2. Every NFA with n states has an equivalent DFA with at most 2^n states.
- C3. Under convention A, Thompson's NFA has at most $2 \times (\#operators + \#operands)$ states.
- C4. Minimising a DFA can change the language it accepts.
- C5. Two DFAs accept the same language if and only if their minimal DFAs are identical up to renaming.
- C6. Two states with identical rows, both accepting or both non-accepting, are equivalent.
- C7. $\{a^n b^n : n \geq 0\}$ is a regular language.
- C8. Longest match always chooses the rule written first.
- C9. Rule order matters only when two rules match the same longest lexeme.
- C10. In flex, the pattern `.` matches a newline.
- C11. A catch-all error rule should be written first in the specification.
- C12. $\varepsilon\text{-closure}(S)$ always contains S .
- C13. Simulating a Thompson NFA on an input of length n costs $O(n|Q|)$.
- C14. The scan-and-backtrack algorithm may read the same character more than once.
- C15. Nested comments can be described by one regular expression.
- C16. Abbreviations in a lexer specification may be recursive.
- C17. The attribute of a string token is its raw source text, quotes included.
- C18. Python's INDENT and DEDENT tokens can be produced by a finite automaton alone.
- C19. A regular language has a unique minimal DFA, but many regular expressions describe it.
- C20. Hopcroft's algorithm and table filling can give different minimal DFAs for the same input.